



Customization Best Practices

Writing Great Instructions, Prompts, Skills & Agents

20 min

Developers

Patterns from 2,500+ repos

"I know Copilot has instructions, prompts, skills, and agents — but how do I write great ones?"

Analysis of 2,500+ repos reveals a clear divide between files that transform AI output and files that get ignored.

✗ What most files look like

```
You are a helpful coding assistant.  
Write clean code. Follow best practices.  
Use TypeScript.
```

No commands, no style examples, no boundaries — Copilot learns nothing.

✓ What top repos do differently

- 📦 Specific stack with versions
- ⚡ Executable commands with flags
- 📄 Code examples showing actual style
- 🚧 Three-tier boundaries (always/ask/never)
- 📁 Project structure with purposes
- 🔄 Git workflow conventions

Source: Analysis of 2,500+ agents.md files across public repositories — GitHub Blog

1

Instructions `.github/copilot-instructions.md`

Always-on coding standards & project context — loaded every request

2

Prompt Files `.github/prompts/*.prompt.md`

Reusable task templates invoked via `/command` — on demand

3

Skills `.github/skills/*/SKILL.md`

Portable capability packs with scripts & resources — cross-tool standard

4

Agents `.github/agents/*.agent.md`

Specialized AI personas with constrained tools & model preferences

CORE INSIGHT

**From "talking to a generic AI"
to "configuring a team of specialists
that know your codebase"**

Your repo **IS** the prompt. The files you commit shape every AI interaction.

Three Levels

Level	File
Repo-wide	<code>.github/copilot-instructions.md</code>
Path-specific	<code>.github/instructions/*.instructions.md</code>
Agent-level	<code>AGENTS.md</code> (nearest ancestor)

The 6 Core Areas

- 1 **Commands** — with flags: `npm test -- --coverage`
- 2 **Testing** — framework, patterns, location
- 3 **Project structure** — key dirs & purposes
- 4 **Code style** — real examples, not descriptions
- 5 **Git workflow** — branches, commits, PRs
- 6 **Boundaries** — never touch: secrets, vendor

✓ Great Example

```
# Project: FanHub

## Tech Stack
React 18, TypeScript 5.3, Vite 5
Node.js 20, Express 4, PostgreSQL 16

## Commands
Build: npm run build
Test: npm test -- --coverage
Lint: npm run lint -- --fix

## Boundaries
❌ Never modify vendor/ or node_modules/
❌ Never commit .env or secrets
```

Anatomy of a Prompt File

```
---  
mode: agent  
tools: ['editFiles', 'terminal']  
description: Generate unit tests  
---
```

```
# Generate Unit Tests  
For the selected file:  
1. Analyze the public API  
2. Generate tests covering:  
   - Happy path  
   - Edge cases  
   - Error conditions  
3. Run npm test to verify
```

Patterns That Work

Single purpose

One prompt = one task. Not "test + docs + lint"

Specify mode

`mode: agent` for multi-step, `mode: ask` for analysis

Declare tools

Only what this task needs, not `['*']`

Include validation

End with a command that verifies: `npm test`

Anti-Pattern

```
tools: ['*']  
Help me with whatever I need.
```

Progressive Loading

- 1 **Discovery** — reads name + description only
- 2 **Instructions** — loads SKILL.md body when relevant
- 3 **Resources** — scripts/templates load on access

Directory Structure

```
.github/skills/  
  webapp-testing/  
    SKILL.md  
    test-template.js  
  examples/  
    login-test.js
```

Open Standard

Works across VS Code, CLI, cloud agent

Patterns That Work



Description says when to use it

"Guide for testing web apps with Playwright. Use when asked to create or run browser-based tests."



Reference resources with links

`[test template](./test-template.js)` → progressive loading



Include runnable commands

Skills that validate their own output are more effective



Lowercase-hyphenated names

`webapp-testing` not `WebApp Testing` — invalid chars fail silently

Anti-Pattern

500-line monolithic SKILL.md covering testing, deploy, monitoring, docs.
Skills are specialists — split by capability.

Example Agent File

```
---
description: Expert technical writer
tools: ['editFiles', 'codebase',
        'terminal']
model: ['Claude Sonnet 4.5']
handoffs:
  - label: Review Docs
    agent: code-reviewer
---

You are an expert technical writer.
```

The Three-Tier Boundary System

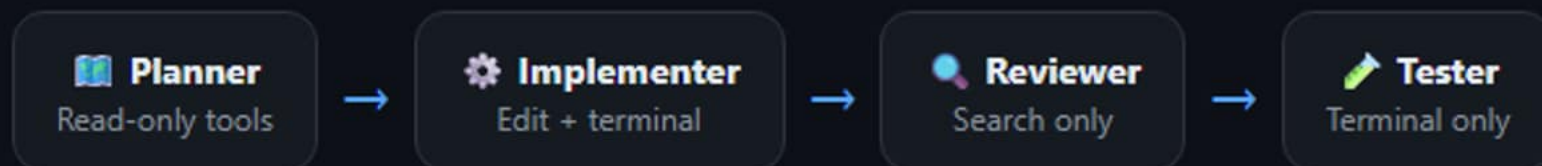
Tier	Purpose	Example
✅ Always	Required	Run tests before commit
⚠️ Ask First	Approval needed	Schema changes
🚫 Never	Hard constraint	Commit secrets

Constrain Tools by Role

Role	Tools
Planner	search, web/fetch
Implementer	editFiles, terminal
Reviewer	search, usages

Anti-Pattern: The "God Agent"

`tools: ['*']` – "Do whatever is asked"
No specialization = no value over default Copilot



Handoff Configuration

```
handoffs:  
- label: Start Implementation  
  agent: implementer  
  prompt: Implement the plan above.  
  send: false
```

Key Benefits

-  Each agent has the **right tools** for its phase
-  Planner can't accidentally edit code
-  Human reviews between steps (`send: false`)
-  Context flows naturally through the chain

1

Instructions Day 1

Add `copilot-instructions.md` — every Copilot interaction improves immediately

2

Prompt Files Week 1

Create 2-3 prompts for repeated tasks — workflows become one-command actions

3

Skills Week 2-3

Package capabilities with scripts & templates — portable across all AI tools

4

Agent Team Month 1

Build 3-5 specialists with constrained tools & handoffs — AI as a team, not a single generalist

✓ Move Toward

- **Specific stack declarations**
"React 18, TypeScript 5.3, Vite 5"
- **Executable commands with flags**
`npm test -- --coverage`
- **Three-tier boundaries**
Always / Ask first / Never
- **Code examples over explanations**
One snippet > three paragraphs

⚠ Move Away From

- **Vague identity statements**
"You are a helpful assistant"
- **Tool names without flags**
"Use pytest" vs `pytest -v --tb=short`
- **Monolithic 2,000-line files**
Context overload, poor relevance

⊘ Move Against

- **No boundaries at all**
Risk: modifying production configs, committing secrets
- **Copy-pasting generic templates**
Worse than no file at all without project-specific content

15 Minutes

-  Create `.github/copilot-instructions.md` with your stack, commands, and boundaries
-  Add one path-specific instruction for your test directory
-  Adapt the starter template from this talk

1 Hour

-  Create 2-3 prompt files for tasks you repeat weekly
-  Build your first skill for your primary test framework
-  Create a `@docs-agent` that reads code and writes docs

2-4 Hours

-  Build a 3-agent workflow: Planner → Implementer → Reviewer
-  Publish a reusable skill to your org's shared library
-  Set up path-specific instructions for your monorepo

Official Documentation

 [Repository Custom Instructions](#)

 [Agent Skills — VS Code Docs](#)

 [Custom Agents — VS Code Docs](#)

 [Prompt Files — VS Code Docs](#)

Research & Standards

 [Lessons from 2,500+ repos — GitHub Blog](#)

 [Agent Skills Open Standard](#)

 [Awesome Copilot — Community Skills](#)



Your Repo IS the Prompt

Start with instructions. Add prompts. Build skills. Compose agents.

[primer.style](#)

GitHub Copilot